

Module 13: Lambdas

A lambda is an implementation of an interface with exactly one abstract method, written without the ceremony of a class.

That definition is worth holding onto. A lambda is not a new kind of value in Java. It is still an object implementing an interface. Module 07 said an interface with a single abstract method can be implemented without writing a class; this module is that sentence made practical.

What You'll Learn

- Lambda syntax, and what the compiler infers for you
- The four method reference forms, including the two that look identical
- Capture, and why a captured local must be effectively final
- The standard functional interfaces and how they compose
- Why `java.util.function` has so many nearly identical types

Coming From Another Language

	Python / JS	Java
Anonymous function	<code>lambda</code> , <code>=></code>	<code>-></code>
Multi-statement body	JS yes, Python no	yes, with braces
Captured variable may change	yes	no , must be effectively final
Type of a lambda	a function object	an interface implementation
Function type	first class	there is none; use an interface

The two rows that matter: Java has **no function type**, so every lambda needs a target interface, and a captured local **cannot be reassigned**.

The Lesson

The same thing, three ways

From `Lambdas.java`:

```
Greeter named = new PoliteGreeter();           // a named class
Greeter anonymous = new Greeter() {           // an anonymous class
    @Override public String greet(String n) { return "Good day, " + n; }
};
Greeter lambda = name -> "Good day, " + name; // a lambda
```

All three produce an object implementing `Greeter`. The lambda omits the interface name, the method name, the modifiers and the parameter type, because the compiler can work all of them out from the target type.

Syntax forms:

```
() -> doSomething()           // no parameters, parentheses required
name -> "hello " + name       // one parameter, parentheses optional
(String name) -> ...          // explicit type when inference needs help
name -> { ...; return x; }     // block body needs braces and return
(a, b) -> a + b               // two parameters always need parentheses
```

A functional interface has exactly one abstract method. Default and static methods do not count, which is why `Comparator` can have dozens of methods and still work as a lambda target. Write `@FunctionalInterface` on your own: it makes the compiler enforce the rule, so adding a second abstract method later fails at the interface rather than at every lambda that used it.

Method references

Four forms, from `MethodReferences.java`:

Form	Example	Equivalent lambda
static	<code>Integer::parseInt</code>	<code>s -> Integer.parseInt(s)</code>
constructor	<code>ArrayList::new</code>	<code>() -> new ArrayList<>()</code>
bound instance	<code>prefix::concat</code>	<code>s -> prefix.concat(s)</code>
unbound instance	<code>String::toUpperCase</code>	<code>s -> s.toUpperCase()</code>

The unbound form is the one people misread. The lambda's first argument becomes the **receiver**, not an argument. That is why `Function<String, String> upper = String::toUpperCase` works even though `toUpperCase` takes no parameters.

Forms three and four look the same and are not:

```
bound, arg "fix":           pre-fix
bound, arg "amble":        pre-amble
unbound, ("pre-", "fix"):   pre-fix
unbound, ("post-", "fix"):  post-fix
```

The bound one always concatenates onto `prefix`. The unbound one takes its receiver from the first argument, so what it concatenates onto changes per call. Tell them apart by what sits left of the `::`: a

variable means bound, a type name means unbound or static.

When not to use one. `words.forEach(System.out::println)` is clearer. But once anything else happens, a lambda reads better. Contorting code so a method reference fits usually makes it worse.

Capture

A captured local must be **final or effectively final**.

Effectively final means you never assign to it after initialisation. You do not write the keyword; the compiler checks the behaviour. `NotEffectivelyFinal.java`:

```
error: local variables referenced from a lambda expression must be final or effectively final
```

Why the rule exists: a lambda can outlive the method that made it. By the time it runs, that stack frame may be gone, so the lambda holds a **copy of the value**, not a reference to the variable. If reassignment were allowed, two questions would have no good answer: does an already-created lambda see the new value, and what happens when two threads assign at once?

The rule is about the variable, not the object. This is fine:

```
List<String> items = new ArrayList<>();
Runnable r = () -> items.add("x");    // the variable never changes
```

Which is also the loophole people use to get around it:

```
int[] counter = {0};
list.forEach(x -> counter[0]++);    // compiles, and is a smell
```

It compiles because the variable never changes. It is a smell because the lambda is being used for mutation, which is what Module 14 exists to replace.

Fields have no such rule. A lambda captures `this` and reads the field through it, so a mutable field works. That also means a stored lambda keeps the whole enclosing object alive, which matters for long-lived listener lists.

this differs between a lambda and an anonymous class:

```
inside a lambda, this is:          Capture
inside an anonymous class, this is: Capture$1
```

The `$1` is a real, separate class. A lambda creates no such thing, so `this` inside it means the enclosing object. This is a behavioural difference, not a style preference.

The standard interfaces

Interface	Shape	Method
<code>Function<T,R></code>	T in, R out	<code>apply</code>
<code>Predicate<T></code>	T in, boolean out	<code>test</code>
<code>Supplier<T></code>	nothing in, T out	<code>get</code>
<code>Consumer<T></code>	T in, nothing out	<code>accept</code>

Plus `BiFunction`, `BiPredicate`, `BiConsumer`, `UnaryOperator<T>` (a `Function<T,T>`) and `BinaryOperator<T>`.

Composition, from `StandardInterfaces.java`:

```
twice.andThen(plusOne) on 5: 11
twice.compose(plusOne) on 5: 12
```

`andThen` means "do me first, then the argument". `compose` means "do the argument first, then me". Getting them the wrong way round is a classic bug whenever the operations are not commutative.

Predicates combine with `and`, `or` and `negate`.

Primitive specialisations exist because `Function<Integer,Integer>` boxes every value.

`IntPredicate`, `ToIntFunction`, `IntUnaryOperator` and their `long` and `double` equivalents avoid it. This is Module 02's boxing cost showing up in API design, and it is why `java.util.function` is so large.

Run It

```
java modules/13-lambdas/examples/Lambdas.java
java modules/13-lambdas/examples/MethodReferences.java
java modules/13-lambdas/examples/Capture.java
java modules/13-lambdas/examples/StandardInterfaces.java

# Fails on purpose.
java modules/13-lambdas/examples/NotEffectivelyFinal.java

./scripts/verify-examples.sh modules/13-lambdas
```

Common Mistakes

Trying to reassign a captured local. Compute the final value first, or use a second variable.

The `int[] box = {0}` trick to accumulate. It compiles. It also says the lambda wants to be a reduction, which Module 14 covers.

Confusing `andThen` with `compose`. Read them aloud: "and then" is second, "compose" wraps around.

Writing `a.length() - b.length()` as a comparator. It overflows for large values. Use `Comparator.comparingInt`.

Expecting `this` in a lambda to be the lambda. There is no lambda object to refer to.

Declaring your own functional interface when a standard one fits. A reader who knows `Function<String,Integer>` has to go and look up your `StringScorer`.

Storing lambdas in long-lived collections without thinking about `this`. The lambda holds the enclosing object, so it cannot be collected.

Key Takeaways

- **A lambda implements a one-abstract-method interface.** Java has no function type.
- `@FunctionalInterface` makes the compiler enforce the rule on interfaces you own.
- **Four method reference forms**, and the unbound one turns the first argument into the receiver.
- **Captured locals must be effectively final**, because the lambda copies the value and may outlive the frame.
- **The rule is about the variable, not the object**, so mutating a captured collection is legal.
- `andThen` and `compose` run in opposite orders.
- **Primitive specialisations exist to avoid boxing.**

Homework

[homework/README.md](#)

Build a small rules engine out of composed predicates and functions, then hit the capture rule on purpose. Reference solution in [solutions/13-lambdas/](#).